TalentFlow Sprint 2 Progress Update and Retrospective

Completion

The system is **22% complete** (41 of 187 development hours). Our timeline planned **43%** by Oct 24, so we are about 40 hours behind. Job postings (4.2.1) are done. The rest of the Job & Application Portal (careers page, application form, resume upload) has not started, and the AI Engine (4.3), planned for Oct 18–24, has not started either.

Development work	Estimated (hrs)	Done (hrs)	Status
3.2.3.1 Hosting, Git & CI/CD	6	6	Done in Sprint 1
4.1 Authentication & Security	25	25	Done in Sprint 1
4.2.1 Job posting create, edit & open/close	10	10	Done in Sprint 2, in review (PR #21)
4.2.2 Public careers page	6	0	Moved to Sprint 3
4.2.3 Application form & resume upload	9	0	Moved to Sprint 3 (database ready, PR #22)
4.3 onward: remaining core modules	71	0	Not started
5. Testing & Quality Assurance	24	0	Not started
6. Deployment & Project Closure	37	0	Not started
Total	**187**	**41**	**22%**

Same method as Sprint 1: system development only, using the PERT expected hours from Project Timeline v2. Sprint 2 finished 10 of the 24 hours planned for it (42%). The planned 43% counts 4.2 (due Oct 17) and 4.3 (due Oct 24).

Completed in Sprint 2

Sprint 2 delivered job postings (4.2.1) and the database and shared pieces for the rest of the portal.

WBS	Task

4.2.1	Job postings API: create, list, view, edit, open/close. Only managers can change postings; visitors only see open ones.
4.2.1.1	Job posting create form with a requirements editor and live preview
4.2.1.2	Edit page and open/close switch (closing asks to confirm); prompt before leaving unsaved changes
4.2.1	Backend unit and e2e tests for every job postings endpoint; frontend tests for the list, form, and requirements editor
4.2.3	Applications, applicant profiles, and resumes tables (migration 0002); resume upload token; shared careers service, public page layout, and resume picker for the Sprint 3 tasks
—	Frontend code brought in line with the README Angular coding guidelines; Prettier set to 80 columns
—	Sprint 2 task document: each member's task, setup steps, and merge plan

Sprint 1 was released from dev to main on Oct 20 (PR #20). Sprint 2 work is in PR #21 (job postings) and PR #22 (database and shared pieces), both waiting for review.

Carried over

WBS	Task	Est. (hrs)	Moves to
4.2.2.1	Public careers page with open listings (responsive)	6	Sprint 3
4.2.3	Application form with validation, and resume upload	9	Sprint 3
—	Test matrix: job postings, careers page, application form, resume upload	—	Sprint 3
3.3	AI proof of concept (prompt design, token and response-time checks)	16	Sprint 3, before the Gemini integration

Sprint 3 plan (Oct 25–31)

Sprint 3 first finishes the portal, then starts the AI Engine (4.3). The portal tasks were assigned on Oct 23 with a task document; each can be done on its own once PR #21 and #22 are merged.

WBS	Task	Owner
—	Review and merge PR #21 and #22 into dev	Lauron, reviewer
4.2.2.1	Public careers page and role page (responsive)	Ng
4.2.3.1	Application API and apply page with validation	Martin
4.2.3.2	Resume upload API, backend tests, resume upload test matrix	Dela Cruz
—	Frontend tests for the shared pieces; careers and apply test matrix	Somuelo
3.3, 4.3	AI proof of concept, then Gemini prompt and API integration, resume text extraction, fit score UI	To assign on Oct 25
—	Sprint 3 progress document and Notion board updates	Martin

Each task is one branch and one pull request into dev.

Retrospective

What went well

- Job postings work end to end: managers create, edit, and open or close postings, and the API, pages, and tests are done.
- The database for applications and resumes is designed and ready, so the remaining portal tasks no longer wait on it. Resumes are stored in the database because the API host deletes uploaded files on every deploy.
- Sprint 3 portal tasks are split so each member’s branch builds and passes its tests on its own, and the branches merge in any order. This fixes Sprint 1’s merge conflicts.
- The frontend now follows the README Angular coding guidelines, and Prettier keeps lines within 80 columns.

What could be improved

- We are further behind: about 40 hours, up from 24. The portal slipped again, and the AI

Engine (4.3) has not started.
- Tasks were assigned late again (Oct 23), even though Sprint 1's action item was to assign them on day one. Most of the sprint's code came from one member.
- Early code did not follow the README coding guidelines (interface instead of type, async/await, 100-column lines) and had to be reworked. Check new code against the guidelines before opening a pull request.
- A local .env was pointing at the live database. It was caught before any test data was written. Each member now uses their own Neon branch for local work.
- Sprint 1 was released to main on Oct 20, three days after the planned Oct 17.

What needs to be done

- ☒ Release Sprint 1 from dev to main (Oct 20)
- ☒ Assign Sprint 2 portal tasks with a task document (Oct 23)
- ☐ Review and merge PR #21 (job postings) and PR #22 (database and shared pieces)
- ☐ Every member: pull dev, run the database migration on their own Neon branch, and start their task
- ☐ Finish the careers page, application form, and resume upload, with test matrices
- ☐ Decide how job postings are deleted: the app closes postings instead, because applications refer to them, but the project requires Delete in its CRUD
- ☐ Bring the Sprint 1 frontend code (login, dashboard, team, auth service) in line with the coding guidelines
- ☐ Assign the AI Engine (4.3) tasks on Oct 25 and start the AI proof of concept (3.3)